

LAPORAN PRAKTIKUM
STRUKTUR DATA
DOUBLE LINKED LIST

Disusun Oleh :

Nama : Nabila Khairunnisa
Nim : 2511531003
Dosen Pengampu : Dr. Wahyudi, S.T, M.T
Asisten Praktikum : M. Galid Avero



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji Syukur atas kehadiran Allah SWT yang telah melimpahkan Rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum struktur data dengan judul “Double Linked List” dengan baik dan tepat waktu. Dalam menyelesaikan laporan ini saya banyak mendapat arahan dan bimbingan, oleh karena itu saya ingin mengucapkan terimakasih kepada

1. Bapak Dr. Wahyudi, S.T, M.T selaku dosen pengampu
2. Uda M. Galid Averro selaku asisten labor
3. Orang tua yang senantiasa mendoakan
4. Teman teman yang selalu memotivasi

Penulis menyadari bahwa laporan ini jauh dari kata sempurna. Oleh sebab itu, penulis sangat membuka diri apabila ada yang ingin memberikan kritikan dan saran yang sifatnya membantu, penulis akan sangat senang menerima. Tujuannya agar untuk kedepannya bisa menyempurnakan laporan.

Padang, 11 Mei 2026

Nabila Khairunnisa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 Deskripsi Praktikum.....	2
2.2 Langkah Langkah Praktikum.....	2
BAB III KESIMPULAN.....	8
3.1 Kesimpulan	8
3.2 Saran	8
DAFTAR PUSTAKA	9
LAPORAN TUGAS PEKAN 4	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Linked list adalah struktur data linier berbentuk rantai simpul di mana setiap simpul menyimpan 2 item, yaitu nilai data dan pointer ke simpul elemen berikutnya. Berbeda dengan array, linked list tidak disimpan secara berurutan di memori. Setiap node dapat berada di lokasi memori yang berbeda, tetapi tetap terhubung melalui pointer.

Double linked list adalah linked list yang setiap node-nya memiliki dua pointer yaitu pointer ke node berikutnya dan pointer ke node sebelumnya. Kelebihan dari Double Linked List adalah bisa bergerak maju dan mundur, penghapusan node lebih mudah, navigasi lebih fleksibel. Namun, Double Linked List juga memiliki kekurangan yaitu membutuhkan memori lebih besar dan struktur lebih kompleks. [1]

Sebuah *linked list* dikatakan kosong apabila isi pointer *head* adalah null. selain itu, nilai pointer *prev* dari *head* selalu null, karena merupakan data pertama. begitu pula dengan pointer *next* dari *tail* yang selalu bernilai null sebagai penanda data terakhir.

Single Linked List dan Double List memiliki perbedaan yaitu Single linked list hanya memiliki satu pointer next, sedangkan double linked list memiliki dua pointer, prev dan next. Double Linked List digunakan ketika diperlukan akses data dari kedua arah dan operasi penyisipan dan penghapusan yang cepat. [2]

1.2 Tujuan Praktikum

1. Mampu memahami konsep dasar Double Linked List.
2. Mampu mengetahui cara kerja hubungan antar node pada Double Linked List melalui pointer next dan prev.
3. Mampu melatih kemampuan dalam membuat program untuk menambah node di awal, akhir dan posisi tertentu pada Double Linked List.
4. Mampu meningkatkan kemampuan dalam melakukan penelusuran data secara maju data mundur.
5. Mampu memahami proses penghapusan node pada berbagai posisi seperti di awal, di akhir maupun di posisi tertentu.

1.3 Manfaat Praktikum

1. Mahasiswa mampu memahami penerapan Double Linked List.
2. Mahasiswa mampu memahami dalam membuat program untuk menambah node di awal, akhir dan posisi tertentu pada Double Linked List.
3. Mahasiswa meningkatkan kemampuan dalam melakukan penelusuran data.
4. Mahasiswa mampu proses penghapusan node pada berbagai posisi seperti di awal, di akhir maupun di posisi tertentu.

BAB II

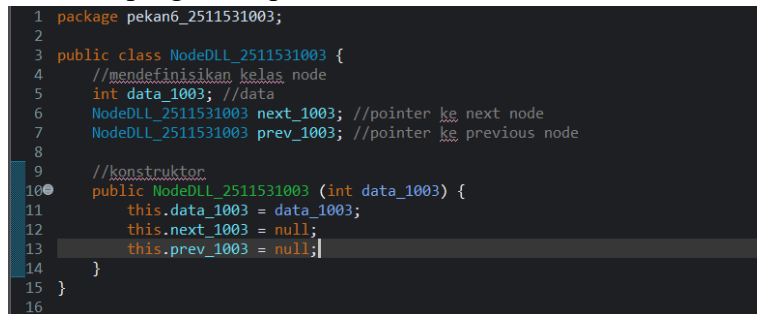
PEMBAHASAN

2.1 Deskripsi Praktikum

Praktikum Struktur data pada pekan ini adalah membuat Single Linked List yang diimplementasikan dengan menggunakan class `NodeSLL_2511531003` sebagai class untuk membuat node pada Single Linked List. Setiap node memiliki data dan pointer next yang digunakan untuk menghubungkan node satu dengan node berikutnya. Selanjutnya dibuat class utama untuk menjalankan operasi dasar pada Single Linked List. Operasi yang dilakukan yaitu menambahkan data, menghapus node pertama, menghapus node terakhir, menghapus node pada posisi tertentu serta menampilkan isi Single Linked List.

2.2 Langkah Langkah Praktikum

- 1) Buatlah package terlebih dahulu dengan mengklik kanan di folder `PrakSD2026_C_2511531003/src`, pilih new dan klik package. Setelah itu beri nama pada package tanpa huruf kapital, karakter khusus serta tanpa “space”. lalu “finish”.
- 2) Klik kanan pada package `pekan5_2511531003` yang sudah dibuat sebelumnya, pilih “New”, lalu pilih class. Buat nama “`NodeDLL_2511531003`” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan sebagai rancangan atau cetakan untuk membuat node pada Double Linked List.
- 3) Tuliskan program seperti berikut.



```
1 package pekan6_2511531003;
2
3 public class NodeDLL_2511531003 {
4     //mendefinisikan kelas node
5     int data_1003; //data
6     NodeDLL_2511531003 next_1003; //pointer ke next node
7     NodeDLL_2511531003 prev_1003; //pointer ke previous node
8
9     //konstruktor
10    public NodeDLL_2511531003 (int data_1003) {
11        this.data_1003 = data_1003;
12        this.next_1003 = null;
13        this.prev_1003 = null;
14    }
15 }
16
```

Gambar 2.2.1 kode program

- `int data_1003` digunakan untuk menyimpan data utama pada node dengan tipe data int.
- `NodeDLL_2511531003 next_1003` berfungsi sebagai pointer atau penunjuk ke node berikutnya.

- *NodeDLL_2511531003 prev_1003* berfungsi sebagai pointer atau penunjuk ke previous berikutnya.
 - *public NodeDLL_2511531003 (int data_1003)* adalah bagian konstruktor. Konstruktor akan dijalankan secara otomatis ketika objek node dibuat.
 - *This.data_1003 = data_1003* berarti data dikirim melalui parameter akan disimpan ke dalam variabel milik class.
 - *this.next_1003 = null* berarti saat node pertama kali dibuat, node tersebut belum menunjuk ke node lain sehingga diatur menjadi null.
 - *this.prev_1003 = null* berarti node belum terhubung ke node sebelumnya.
- 4) Buatlah class kedua dengan nama “InsertDLL_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan untuk mengatur proses insert data pada DLL.
- 5) Tuliskan program seperti berikut.

```

1 package pekan_2511531003;
2
3 public class InsertDLL {
4     //menambahkan node di awal DLL
5     static NodeDLL_2511531003 insertBegin (NodeDLL_2511531003 head_1003, int data) {
6         //buat node baru
7         NodeDLL_2511531003 new_node = new NodeDLL_2511531003 (data);
8         //jadikan pointer nextnya head
9         new_node.next_1003 = head_1003;
10        //jadikan pointer prev head ke new node
11        if (head_1003 != null) {
12            head_1003.prev_1003 = new_node;
13        }
14        return new_node;
15    }
16    //fungsi menambahkan node di akhir
17    public static NodeDLL_2511531003 insertEnd (NodeDLL_2511531003 head_1003, int newData) {
18        //buat node baru
19        NodeDLL_2511531003 newNode_1003 = new NodeDLL_2511531003 (newData);
20        //jika dll null jadikan head
21        if (head_1003 == null) {
22            head_1003 = newNode_1003;
23        }
24        else {
25            NodeDLL_2511531003 curr_1003 = head_1003;
26            while (curr_1003.next_1003 != null) {
27                curr_1003 = curr_1003.next_1003;
28            }
29            curr_1003.next_1003 = newNode_1003;
30            newNode_1003.prev_1003 = curr_1003;
31        }
32        return head_1003;
33    }
34 }
35 //fungsi menambahkan node di posisi tertentu
36 public static NodeDLL_2511531003 insertAtPosition (NodeDLL_2511531003 head_1003, int pos_1003, int new_data_1003) {
37     //buat node baru
38     NodeDLL_2511531003 new_node = new NodeDLL_2511531003 (new_data_1003);
39     if (pos_1003 == 1) {
40         new_node.next_1003 = head_1003;
41         if (head_1003 != null) {
42             head_1003.prev_1003 = new_node;
43         }
44         head_1003 = new_node;
45         return head_1003;
46     }
47     NodeDLL_2511531003 curr = head_1003;
48     for (int i = 1; i < pos_1003 - 1 && curr != null; ++i) {
49         curr = curr.next_1003;
50         if (curr == null) {
51             System.out.println ("Posisi tidak ada");
52             return head_1003;
53         }
54         new_node.prev_1003 = curr;
55         new_node.next_1003 = curr.next_1003;
56         curr.next_1003 = new_node;
57         if (new_node.next_1003 != null) {
58             new_node.next_1003.prev_1003 = new_node;
59         }
60         return head_1003;
61     }
62     public static void printList (NodeDLL_2511531003 head) {
63         NodeDLL_2511531003 curr = head;
64         while (curr != null) {
65             System.out.print (curr.data_1003 + " <-> ");
66             curr = curr.next_1003;
67         }
68         System.out.println();
69     }
70 }

```

```

67 public static void main(String[] args) {
68     //membuat dll 2 <-> 3 <-> 5
69     NodeDLL_2511531003 head = new NodeDLL_2511531003 (2);
70     head.next_1003 = new NodeDLL_2511531003 (3);
71     head.next_1003.prev_1003 = head;
72     head.next_1003.next_1003 = new NodeDLL_2511531003 (5);
73     head.next_1003.next_1003.prev_1003 = head.next_1003;
74     //cetak DLL awal
75     System.out.print ("DLL Awal: ");
76     printList (head);
77     //tambah 1 di awal
78     head = insertBegin (head, 1);
79     System.out.print(
80         "simpul 1 ditambah di awal: ");
81     printList(head);
82     //tambah 6 di akhir
83     System.out.print(
84         "simpul 6 ditambah di akhir: ");
85     int data = 6;
86     head = insertEnd (head, data);
87     printList(head);
88     //menambah node 4 di posisi 4
89     System.out.print("tambah node 4 di posisi 4: ");
90     int data2 = 4;
91     int pos = 4;
92     head = insertAtPosition (head, pos, data2);
93     printList(head);
94 }
95 }
96

```

Gambar 2.2.2 Kode Program

- *insertBegin()* berfungsi untuk menambahkan node baru di awal DLL.
- *insertEnd ()* berfungsi untuk menambahkan node baru di akhir DLL.
- *insertAtPosition()* berfungsi untuk menambahkan node baru pada posisi tertentu.
- *printList()* fungsinya untuk mencetak isi DLL dari awal sampai akhir.
- Method *main()* berfungsi sebagai program utama untuk menjalankan program.

- 6) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```

<terminated> InsertDLL [Java Application] C:\Users\lenovo\.p2\pool\plugins\org.eclipse.justj
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

Gambar 2.2.3 Output program

- 7) Class ketiga dengan nama “PenelusuranDLL_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi untuk melakukan traversal (penelusuran) pada Double Linked List, baik maju (forward) maupun mundur (backward).

8) Tuliskan program seperti berikut.

```

1 package pekan6_2511531003;
2
3 public class PenelusuranDLL_2511531003 {
4     //fungsi penelusuran maju
5     static void forwardTraversal (NodeDLL_2511531003 head) {
6         //memulai penelusuran dari head
7         NodeDLL_2511531003 curr = head;
8         //lanjutkan sampai akhir
9         while (curr != null) {
10            //print data
11            System.out.print(curr.data_1003 + " <-> ");
12            //pindah ke node berikutnya
13            curr = curr.next_1003;
14        }
15        //print spasi
16        System.out.println();
17    }
18    //fungsi penelusuran mundur
19    static void backwardTraversal (NodeDLL_2511531003 tail) {
20        //mulai dari akhir
21        NodeDLL_2511531003 curr = tail;
22        //lanjutkan sampai head
23        while (curr != null) {
24            //cetak data
25            System.out.print(curr.data_1003 + " <-> ");
26            //pindah ke node sebelumnya
27            curr = curr.prev_1003;
28        }
29        //print spasi
30        System.out.println();
31    }
32
33    public static void main(String[] args) {
34        //cetak DLL
35        NodeDLL_2511531003 head = new NodeDLL_2511531003 (1);
36        NodeDLL_2511531003 second = new NodeDLL_2511531003 (2);
37        NodeDLL_2511531003 third = new NodeDLL_2511531003 (3);
38
39        head.next_1003 = second;
40        second.prev_1003 = head;
41        second.next_1003 = third;
42        third.prev_1003 = second;
43
44        System.out.println("Penelusuran maju: ");
45        forwardTraversal(head);
46
47        System.out.println("Penelusuran mundur: ");
48        backwardTraversal(third);
49    }
50 }

```

Gambar 2.2.4 Kode Program

- *forwardTraversal()* berfungsi untuk menelusuri DLL dari awal ke akhir.
- *backwardTraversal()* berfungsi untuk menelusuri DLL dari belakang ke depan.
- Method *main()* berfungsi untuk menjalankan program.

9) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```

<terminated> PenelusuranDLL_2511531003 [J
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

Gambar 2.2.5 Output Program

- 10) Class ke empat dengan nama “HapusDLL_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi untuk menghapus node pada Double Linked List baik di awal, di akhir maupun di posisi tertentu.

- 11) Tuliskan program seperti berikut

```
1 package pekan6_2511531003;
2
3 public class HapusDLL_2511531003 {
4     //fungsi menghapus node awal
5     public static NodeDLL_2511531003 delHead (NodeDLL_2511531003 head) {
6         if (head == null) {
7             return null;
8         }
9         NodeDLL_2511531003 temp = head;
10        head = head.next_1003;
11        if (head != null) {
12            head.prev_1003 = null;
13        }
14        return head;
15    }
16    //fungsi menghapus di akhir
17    public static NodeDLL_2511531003 delLast (NodeDLL_2511531003 head) {
18        if (head == null) {
19            return null;
20        }
21        if (head.next_1003 == null) {
22            return null;
23        }
24        NodeDLL_2511531003 curr = head;
25        while (curr.next_1003 != null) {
26            curr = curr.next_1003;
27        }
28        //update pointer previous node
29        if (curr.prev_1003 != null) {
30            curr.prev_1003.next_1003 = null;
31        }
32        return head;
33    }
34
35    //fungsi menghapus node posisi tertentu
36    public static NodeDLL_2511531003 delPos (NodeDLL_2511531003 head, int pos) {
37        //jika DLL kosong
38        if (head == null) {
39            return head;
40        }
41        NodeDLL_2511531003 curr = head;
42        //telusuri sampai ke node yang akan dihapus
43        for (int i = 1; curr != null && i < pos; ++i) {
44            curr = curr.next_1003;
45        }
46        //jika posisi tidak ditemukan
47        if (curr == null) {
48            return head;
49        }
50        //update pointer
51        if (curr.prev_1003 != null) {
52            curr.prev_1003.next_1003 = curr.next_1003;
53        }
54        if (curr.next_1003 != null) {
55            curr.next_1003.prev_1003 = curr.prev_1003;
56        }
57        //jika yang dihapus head
58        if (head == curr) {
59            head = curr.next_1003;
60        }
61        return head;
62    }
63
64    //fungsi mencetak DLL
65    public static void printList (NodeDLL_2511531003 head) {
66        NodeDLL_2511531003 curr = head;
67        while (curr != null) {
68            System.out.print(curr.data_1003 + " ");
69            curr = curr.next_1003;
70        }
71        System.out.println();
72    }
73
74    public static void main(String[] args) {
75        // buat sebuah DLL
76        NodeDLL_2511531003 head = new NodeDLL_2511531003(1);
77        head.next_1003 = new NodeDLL_2511531003(2);
78        head.next_1003.prev_1003 = head;
79        head.next_1003.next_1003 = new NodeDLL_2511531003(3);
80        head.next_1003.next_1003.prev_1003 = head.next_1003;
81        head.next_1003.next_1003.next_1003 = new NodeDLL_2511531003(4);
82        head.next_1003.next_1003.next_1003.prev_1003 = head.next_1003.next_1003;
83        head.next_1003.next_1003.next_1003.next_1003 = new NodeDLL_2511531003(5);
84        head.next_1003.next_1003.next_1003.next_1003.prev_1003 = head.next_1003.next_1003.next_1003;
85
86        System.out.print("DLL Awal: ");
87        printList(head);
88
89        System.out.print("Setelah head dihapus: ");
90        head = delHead(head);
91        printList(head);
92
93        System.out.print("Setelah node terakhir dihapus: ");
94        head = delLast(head);
95        printList(head);
96
97        System.out.print("menghapus node ke 2: ");
98        head = delPos(head, 2);
99        printList(head);
100    }
101 }
```

Gambar 2.2.6 Kode Program

- *delHead* untuk menghapus node pertama pada linked list.
- *delLast* berfungsi untuk menghapus node terakhir.
- *delPos* berfungsi untuk menghapus node pada posisi tertentu.
- *printList* berfungsi untuk mencetak isi DLL.
- *main* berfungsi untuk menjalankan program.

12) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> HapusDLL_2511531003 [Java Application] C:\Users\le
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
menghapus node ke 2: 2 4
```

Gambar 2.2.7 Output Program

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List memungkinkan setiap node terhubung ke node berikutnya melalui pointer next dan ke node sebelumnya melalui pointer prev, sehingga traversal dapat dilakukan dua arah yaitu maju dan mundur. Pada praktikum ini, terdapat beberapa operasi dasar pada DLL yaitu penambahan node, penelusuran data secara forward dan backward, serta penghapusan node.

3.2 Saran

Untuk menambah pengembangan praktikum di masa mendatang disarankan untuk dapat mengembangkan fitur yang lebih lengkap sehingga pemahaman terhadap implementasi struktur data menjadi lebih mendalam.

DAFTAR PUSTAKA

- [1] dianaape, "Linked List: Single, Double, dan Circular Linked List," Telkom University Jakarta, 13 Februari 2026. [Online]. Available: <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/>. [Accessed 13 Mei 2026].
- [2] admin, "Double Linked List: Panduan Lengkap untuk Pemula," definisi.ac.id, 6 Februari 2024. [Online]. Available: <https://definisi.ac.id/double-linked-list/>. [Accessed 13 Mei 2026].

LAPORAN TUGAS PRAKTIKUM PEKAN 5

1. KODE PROGRAM

//Class 1 : Lagu_2511531003

package pekan6_2511531003;

```
public class Lagu_2511531003 {
    private String judul_1003;
    private String penyanyi_1003;
    Lagu_2511531003 next_1003;
    Lagu_2511531003 prev_1003;

    public Lagu_2511531003(String judul_1003, String penyanyi_1003) {
        this.judul_1003 = judul_1003;
        this.penyanyi_1003 = penyanyi_1003;
        this.next_1003 = null;
        this.prev_1003 = null;
    }

    public String getJudul_1003() {
        return judul_1003;
    }

    public void setJudul_1003(String judul_1003) {
        this.judul_1003 = judul_1003;
    }

    public String getPenyanyi_1003() {
        return penyanyi_1003;
    }

    public void setPenyanyi_1003(String penyanyi_1003) {
        this.penyanyi_1003 = penyanyi_1003;
    }
}
```

//Class 2 : Musik_2511531003

```
package pekan6_2511531003;

import java.util.Scanner;

public class Musik_2511531003 {

    private Lagu_2511531003 head_1003;

    private Lagu_2511531003 tail_1003;


    public Musik_2511531003() {

        head_1003 = null;

        tail_1003 = null;

    }


    // Menambahkan lagu baru di akhir playlist

    public void tambahLagu_1003(String judul_1003, String penyanyi_1003) {

        Lagu_2511531003 laguBaru_1003 = new Lagu_2511531003(judul_1003,
penyanyi_1003);

        if (head_1003 == null) {

            head_1003 = laguBaru_1003;

            tail_1003 = laguBaru_1003;

        } else {

            tail_1003.next_1003 = laguBaru_1003;

            laguBaru_1003.prev_1003 = tail_1003;

            tail_1003 = laguBaru_1003;

        }

    }

}
```

```

        System.out.println("Lagu berhasil ditambahkan!");
    }

    // Menghapus lagu pertama
    public void hapusLaguAwal_1003() {
        if (head_1003 == null) {
            System.out.println("Playlist masih kosong!");
        } else if (head_1003 == tail_1003) {
            System.out.println("Lagu \"" + head_1003.getJudul_1003() + "\" berhasil
dihapus!");

            head_1003 = null;
            tail_1003 = null;
        } else {
            System.out.println("Lagu \"" + head_1003.getJudul_1003() + "\" berhasil
dihapus!");

            head_1003 = head_1003.next_1003;
            head_1003.prev_1003 = null;
        }
    }

    // Menampilkan playlist dari awal ke akhir
    public void tampilMaju_1003() {
        if (head_1003 == null) {
            System.out.println("Playlist masih kosong!");
            return;
        }
    }

```

```

Lagu_2511531003 curr_1003 = head_1003;
int nomor_1003 = 1;

System.out.println("\nDaftar Playlist Maju:");
while (curr_1003 != null) {
    System.out.println(nomor_1003 + ". Judul: " + curr_1003.getJudul_1003()
        + " | Penyanyi: " + curr_1003.getPenyanyi_1003());
    curr_1003 = curr_1003.next_1003;
    nomor_1003++;
}
}

// Menampilkan playlist dari akhir ke awal
public void tampilMundur_1003() {
    if (tail_1003 == null) {
        System.out.println("Playlist masih kosong!");
        return;
    }

    Lagu_2511531003 curr_1003 = tail_1003;
    int nomor_1003 = 1;

    System.out.println("\nDaftar Playlist Mundur:");
    while (curr_1003 != null) {
        System.out.println(nomor_1003 + ". Judul: " + curr_1003.getJudul_1003()
            + " | Penyanyi: " + curr_1003.getPenyanyi_1003());
        curr_1003 = curr_1003.prev_1003;
    }
}

```

```

        nomor_1003++;
    }
}

// Mencari lagu berdasarkan judul
public void cariLagu_1003(String judulCari_1003) {
    if (head_1003 == null) {
        System.out.println("Playlist masih kosong!");
        return;
    }

    Lagu_2511531003 curr_1003 = head_1003;
    boolean ditemukan_1003 = false;

    while (curr_1003 != null) {
        if (curr_1003.getJudul_1003().equalsIgnoreCase(judulCari_1003)) {
            System.out.println("Lagu ditemukan!");
            System.out.println("Judul   : " + curr_1003.getJudul_1003());
            System.out.println("Penyanyi : " + curr_1003.getPenyanyi_1003());
            ditemukan_1003 = true;
            break;
        }
        curr_1003 = curr_1003.next_1003;
    }

    if (!ditemukan_1003) {

```

```
        System.out.println("Lagu dengan judul \"" + judulCari_1003 + "\" tidak  
ditemukan!");
```

```
    }  
}
```

```
public static void main(String[] args) {
```

```
    Scanner input_1003 = new Scanner(System.in);
```

```
    Musik_2511531003 playlist_1003 = new Musik_2511531003();
```

```
    int pilihan_1003;
```

```
    do {
```

```
        System.out.println("\n=== Playlist Musik NIM: 2511531003 ===");
```

```
        System.out.println("1. Tambah Lagu");
```

```
        System.out.println("2. Hapus Lagu Pertama");
```

```
        System.out.println("3. Lihat Playlist (Maju)");
```

```
        System.out.println("4. Lihat Playlist (Mundur)");
```

```
        System.out.println("5. Cari Lagu");
```

```
        System.out.println("6. Keluar");
```

```
        System.out.print("Pilihan: ");
```

```
        pilihan_1003 = input_1003.nextInt();
```

```
        input_1003.nextLine();
```

```
        switch (pilihan_1003) {
```

```
            case 1:
```

```
                System.out.print("Judul: ");
```

```
                String judul_1003 = input_1003.nextLine();
```

```
System.out.print("Penyanyi: ");  
String penyanyi_1003 = input_1003.nextLine();  
  
playlist_1003.tambahLagu_1003(judul_1003, penyanyi_1003);  
break;
```

case 2:

```
playlist_1003.hapusLaguAwal_1003();  
break;
```

case 3:

```
playlist_1003.tampilMaju_1003();  
break;
```

case 4:

```
playlist_1003.tampilMundur_1003();  
break;
```

case 5:

```
System.out.print("Masukkan judul lagu yang dicari: ");  
String cari_1003 = input_1003.nextLine();  
playlist_1003.cariLagu_1003(cari_1003);  
break;
```

case 6:

```
System.out.println("Program selesai. Terima kasih!");
```

```

        break;

        default:

            System.out.println("Pilihan tidak valid!");

            break;

    }

} while (pilihan_1003 != 6);

input_1003.close();

}

}

```

2. SCREENSHOT CODE PROGRAM

- **Class 1 : Lagu_2511531003**



```

1 package pekar_2511531003;
2
3 public class Lagu_2511531003 {
4     private String judul_1003;
5     private String penyanyi_1003;
6     Lagu_2511531003 next_1003;
7     Lagu_2511531003 prev_1003;
8
9     public Lagu_2511531003(String judul_1003, String penyanyi_1003) {
10         this.judul_1003 = judul_1003;
11         this.penyanyi_1003 = penyanyi_1003;
12         this.next_1003 = null;
13         this.prev_1003 = null;
14     }
15
16     public String getJudul_1003() {
17         return judul_1003;
18     }
19
20     public void setJudul_1003(String judul_1003) {
21         this.judul_1003 = judul_1003;
22     }
23
24     public String getPenyanyi_1003() {
25         return penyanyi_1003;
26     }
27
28     public void setPenyanyi_1003(String penyanyi_1003) {
29         this.penyanyi_1003 = penyanyi_1003;
30     }
31 }

```

- **Class 2 : Musik_2511531003**

```

1 package pekan6_2511531003;
2 import java.util.Scanner;
3 public class Musik_2511531003 {
4     private Lagu_2511531003 head_1003;
5     private Lagu_2511531003 tail_1003;
6
7     public Musik_2511531003() {
8         head_1003 = null;
9         tail_1003 = null;
10    }
11
12    // Menambahkan lagu baru di akhir playlist
13    public void tambahLagu_1003(String judul_1003, String penyanyi_1003) {
14        Lagu_2511531003 laguBaru_1003 = new Lagu_2511531003(judul_1003, penyanyi_1003);
15
16        if (head_1003 == null) {
17            head_1003 = laguBaru_1003;
18            tail_1003 = laguBaru_1003;
19        } else {
20            tail_1003.next_1003 = laguBaru_1003;
21            laguBaru_1003.prev_1003 = tail_1003;
22            tail_1003 = laguBaru_1003;
23        }
24        System.out.println("Lagu berhasil ditambahkan!");
25    }
26
27
28    // Menghapus lagu pertama
29    public void hapusLaguAwal_1003() {
30        if (head_1003 == null) {
31            System.out.println("Playlist masih kosong!");
32        } else if (head_1003 == tail_1003) {
33            System.out.println("Lagu \"\" + head_1003.getJudul_1003() + "\" berhasil dihapus!");
34            head_1003 = null;
35            tail_1003 = null;
36        } else {
37            System.out.println("Lagu \"\" + head_1003.getJudul_1003() + "\" berhasil dihapus!");
38            head_1003 = head_1003.next_1003;
39            head_1003.prev_1003 = null;
40        }
41    }
42
43    // Menampilkan playlist dari awal ke akhir
44    public void tampilMaju_1003() {
45        if (head_1003 == null) {
46            System.out.println("Playlist masih kosong!");
47            return;
48        }
49
50        Lagu_2511531003 curr_1003 = head_1003;
51        int nomor_1003 = 1;
52
53        System.out.println("\nDaftar Playlist Maju:");
54        while (curr_1003 != null) {
55            System.out.println(nomor_1003 + ". Judul: " + curr_1003.getJudul_1003()
56                + " | Penyanyi: " + curr_1003.getPenyanyi_1003());
57            curr_1003 = curr_1003.next_1003;
58            nomor_1003++;
59        }
60    }
61
62    // Menampilkan playlist dari akhir ke awal
63    public void tampilMundur_1003() {
64        if (tail_1003 == null) {
65            System.out.println("Playlist masih kosong!");
66            return;
67        }
68
69        Lagu_2511531003 curr_1003 = tail_1003;
70        int nomor_1003 = 1;
71
72        System.out.println("\nDaftar Playlist Mundur:");
73        while (curr_1003 != null) {
74            System.out.println(nomor_1003 + ". Judul: " + curr_1003.getJudul_1003()
75                + " | Penyanyi: " + curr_1003.getPenyanyi_1003());
76            curr_1003 = curr_1003.prev_1003;
77            nomor_1003++;
78        }
79    }
80
81    // Mencari lagu berdasarkan judul
82    public void cariLagu_1003(String judulCari_1003) {
83        if (head_1003 == null) {
84            System.out.println("Playlist masih kosong!");
85            return;
86        }

```

```

88     lagu_2511531003 curr_1003 = head_1003;
89     boolean ditemukan_1003 = false;
90
91     while (curr_1003 != null) {
92         if (curr_1003.getJudul_1003().equalsIgnoreCase(judulCari_1003)) {
93             System.out.println("Lagu ditemukan!");
94             System.out.println("Judul : " + curr_1003.getJudul_1003());
95             System.out.println("Penyanyi : " + curr_1003.getPenyanyi_1003());
96             ditemukan_1003 = true;
97             break;
98         }
99         curr_1003 = curr_1003.next_1003;
100     }
101
102     if (!ditemukan_1003) {
103         System.out.println("Lagu dengan judul \"" + judulCari_1003 + "\" tidak ditemukan!");
104     }
105 }
106
107 public static void main(String[] args) {
108     Scanner input_1003 = new Scanner(System.in);
109     Musik_2511531003 playlist_1003 = new Musik_2511531003();
110
111     int pilihan_1003;
112
113     do {
114         System.out.println("\n=== Playlist Musik NIM: 2511531003 ===");
115         System.out.println("1. Tambah Lagu");
116         System.out.println("2. Hapus Lagu Pertama");
117         System.out.println("3. Lihat Playlist (Maju)");
118         System.out.println("4. Lihat Playlist (Mundur)");
119         System.out.println("5. Cari Lagu");
120         System.out.println("6. Keluar");
121         System.out.print("Pilihan: ");
122
123         pilihan_1003 = input_1003.nextInt();
124         input_1003.nextLine();
125
126         switch (pilihan_1003) {
127             case 1:
128                 System.out.print("Judul: ");
129                 String judul_1003 = input_1003.nextLine();
130
131                 System.out.print("Penyanyi: ");
132                 String penyanyi_1003 = input_1003.nextLine();
133
134                 playlist_1003.tambahLagu_1003(judul_1003, penyanyi_1003);
135                 break;
136
137             case 2:
138                 playlist_1003.hapusLaguAwal_1003();
139                 break;
140
141             case 3:
142                 playlist_1003.tampilMaju_1003();
143                 break;
144
145             case 4:
146                 playlist_1003.tampilMundur_1003();
147                 break;
148
149             case 5:
150                 System.out.print("Masukkan judul lagu yang dicari: ");
151                 String cari_1003 = input_1003.nextLine();
152                 playlist_1003.cariLagu_1003(cari_1003);
153                 break;
154
155             case 6:
156                 System.out.println("Program selesai. Terima kasih!");
157                 break;
158
159             default:
160                 System.out.println("Pilihan tidak valid!");
161                 break;
162         }
163     } while (pilihan_1003 != 6);
164
165     input_1003.close();
166 }
167

```

3. OUTPUT

```

Musik 2511531003 [Java Application] C:\Users\lenovo\p2\pool\plugins\org.eclipse.justj.open
=== Playlist Musik NIM: 2511531003 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Rosalia - Mio Cristop Diamanti
Penyanyi: Rosalia
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531003 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan:

```

4. PENJELASAN TUGAS

Pada tugas pekan 6 ini, diperintahkan untuk membuat sebuah program dengan implementasi Double Linked List. Pada tugas kali ini adalah simulasi playlist lagu sederhana. Program yang dibuat digunakan untuk mengelola daftar lagu, dimana setiap lagu direpresentasikan sebagai sebuah node yang memiliki pointer ke lagu berikutnya (next) dan pointer ke lagu sebelumnya (prev). dengan penggunaan Double Linked List, pengguna dapat melakukan penelusuran data dari dua arah, yaitu dari awal ke akhir maupun dari akhir ke awal. Program ini terdiri dari 2 class yaitu class `lagu_2511531003` dan class `music_2511531003`.

Pada kelas **Lagu_2511531003** merupakan kelas yang digunakan sebagai node dalam Double Linked List. Di dalam kelas ini terdapat atribut *judul_1003*, *penyanyi_1003*, *next_1003*, dan *prev_1003*. Selain itu, class ini memiliki constructor untuk menginisialisasi data lagu ketika objek dibuat, serta method getter dan setter yang digunakan untuk mengambil dan mengubah nilai atribut.

Pada kelas **Musik_2511531003** merupakan kelas driver yang digunakan untuk menjalankan program dan mengelola playlist lagu. Kelas ini berisi atribut *head_1003* sebagai penanda node pertama playlist, *tail_1003* sebagai penunjuk node terakhir playlist. Pada kelas ini terdapat beberapa method seperti *tambahLagu_1003()* untuk menambahkan lagu baru ke bagian akhir playlist, *hapusLaguAwal_1003()* untuk menghapus lagu pertama dalam playlist, *tampilMaju_1003()* untuk menampilkan seluruh daftar lagu dari awal ke akhir, *tampilMundur_1003()* untuk menampilkan daftar lagu dari akhir ke awal dan *cariLagu_1003()* untuk mencari lagu berdasarkan judul.